Devsecops Assignment 2

Part A – Multi-User Simulation:

```
root@Khronicles-PC:~# useradd -m -s /bin/bash intern_a && passwd intern_a
New password:
Retype new password:
passwd: password updated successfully
root@Khronicles-PC:~# useradd -m -s /bin/bash dev_user && passwd dev_user
New password:
Retype new password:
passwd: password updated successfully
root@Khronicles-PC:~# passwd dev
passwd: user 'dev' does not exist
root@Khronicles-PC:~# passwd dev_user
New password:
Retype new password:
passwd: password updated successfully
root@Khronicles-PC:~# useradd -m -s /bin/bash ops_user && passwd ops_user
New password:
Retype new password:
passwd: password updated successfully
root@Khronicles-PC:~# groupadd project_team
root@Khronicles-PC:~# usermod -aG project_team intern_a
root@Khronicles-PC:~# usermod -aG project_team dev_user
root@Khronicles-PC:~# usermod -aG project_team ops_user
root@Khronicles-PC:~# groups intern_a
intern_a : intern_a project_team
root@Khronicles-PC:~# id intern_a
uid=1009(intern_a) gid=1009(intern_a) groups=1009(intern_a),1016(project_team)
root@Khronicles-PC:~#
```

User Creation Commands:

useradd –m –s /bin/bash intern_a && passwd intern_a

useradd –m –s /bin/bash dev_user && passwd dev_user

useradd –m –s /bin/bash ops_user && passwd ops_user

Group Membership Verification:

Command:

To verify the group membership, the following commands were used:

groups intern_a or id intern_a

Part B – Reproduce the Incident

```
root@Khronicles-PC:~# chown 2775 :project_team
chown: cannot access ':project_team': No such file or directory
root@Khronicles-PC:~# chown  :project_team /project_team
root@Khronicles-PC:~# chmod 2775 /project_team
root@Khronicles-PC:~# chown :project_team /project_team/shared_design.doc
root@Khronicles-PC:~# chmod 664 /project_team/shared_design.doc
root@Khronicles-PC:~# su intern_a
intern_a@Khronicles-PC:/root$ rm /project_team/shared_design.doc
intern_a@Khronicles-PC:/root$ ls /project_team/
```

Part C – Prevent Recurrence.

```
root@Khronicles-PC:~# touch /project_team/shared_design.doc
root@Khronicles-PC:~# ls /project_team/
shared_design.doc
```

```
root@Khronicles-PC:~# chown :project_team /project_team/shared_design.doc
root@Khronicles-PC:~# chmod 664 /project_team/shared_design.doc
```

```
root@Khronicles-PC:~# sudo chattr +a /project_team/shared_design.doc
root@Khronicles-PC:~# sudo chattr +i /project_team/shared_design.doc
```

```
root@Khronicles-PC:~# su intern_a
intern_a@Khronicles-PC:/root$ rm /project_team/shared_design.doc
rm: cannot remove '/project_team/shared_design.doc': Operation not permitted
intern_a@Khronicles-PC:/root$
```

Explanation:

1. In part B, the intern was able to delete the file from the directory due to the fact that the permission (2775) on the directory allows users of that group to be able to delete files, even if they weren't the owner. In the part C after applying the attribute to the file, it made it impossible to for deletion.
2. The reason why permission wasn't sufficient is because, permissions controls how users read, write and execute file and directories. However, these alone do not prevent deletion hence the need to apply attributes which tends to restrict actions like deletions.
3. Difference between attributes and permissions: Permissions control how files or directories are accessed at user/group/other level. While attributes apply special flags at the file system level (immutable, append-only).
4. In real production, if these restrictions are not implement, it can lead to the either accidental or malicious deletion of critical files.

Scenario 2 – Log Overwrite Incident

```
root@Khronicles-PC:~# mkdir -p /logs
root@Khronicles-PC:~# seq 1 100 > /logs/app.log
root@Khronicles-PC:~# wc -l /logs/app.log
100 /logs/app.log
root@Khronicles-PC:~# echo "Accidental overwrite occurred" > /logs/app.log
root@Khronicles-PC:~# wc -l /logs/app.log
1 /logs/app.log
root@Khronicles-PC:~# chmod u-w /logs/app.log
root@Khronicles-PC:~# chattr +a /logs/app.log
root@Khronicles-PC:~# mkdir -p /archive
root@Khronicles-PC:~# cp -p /logs/app.log /archive
root@Khronicles-PC:~#
```

Explanation:

1. Overwriting destroys historical records needed for incident investigation. When logs are overwritten it makes hard for security team to not be able to trace events, hence making accountability impossible.
2. Metadata shows when events occurred. It helps identify who generated or modified logs.
3. DevSecOps forensics significance: Metadata reveals unauthorized access attempts. Immutable logs allow accurate reconstruction of attacks.

Scenario 3 – Permission and Ownership Drift

```
root@Khronicles-PC:~# mkdir -p /dirA
root@Khronicles-PC:~# touch /dirA/file1 /dirA/file2
root@Khronicles-PC:~# chmod 644 /dirA/file1
root@Khronicles-PC:~# chmod 600 /dirA/file2
root@Khronicles-PC:~#
root@Khronicles-PC:~#
root@Khronicles-PC:~# chown intern_a /dirA/file1
root@Khronicles-PC:~# chown dev_user /dirA/file2
root@Khronicles-PC:~#
```

```
root@Khronicles-PC:~# chgrp project_team /dirA/file1
root@Khronicles-PC:~# chgrp project_team /dirA/file2
root@Khronicles-PC:~#
root@Khronicles-PC:~# mkdir -p /dirB
root@Khronicles-PC:~#
root@Khronicles-PC:~# cp /dirA/file1 /dirB
root@Khronicles-PC:~#
root@Khronicles-PC:~# cp -p /dirA/file2 /dirB
root@Khronicles-PC:~#
root@Khronicles-PC:~# ls -al dirB
ls: cannot access 'dirB': No such file or directory
root@Khronicles-PC:~# ls -al /dirB
total 8
drwxr-xr-x  2 root     root         4096 Mar  3 16:18 .
drwxr-xr-x 27 root     root         4096 Mar  3 16:17 ..
-rw-r--r--  1 root     root            0 Mar  3 16:17 file1
-rw-------  1 dev_user project_team    0 Mar  3 16:05 file2
root@Khronicles-PC:~#
```

Explanation:

From the observation, it was noted that when a copy was done for the file1 without adding the preservation flag, the copied file in dirB lost its ownership and group, hence defaulting to the user performing the copy. On the other hand, when file2 was copied to dirB with the preservation flag –p, the copied file maintained its ownership and group.

Scenario 4: Relative Path Deployment Failure

Why relative paths caused issues:

Relative paths depend on the current working directory. If the script is executed from a different location, the path resolution breaks, leading to deployment failures. Absolute paths are deterministic, they always point to the same location, independent of where the script is run.

Scenario 5: Monitoring Failure After Log Cleanup

```
root@Khronicles-PC:~# echo "Log entry 1" > app.log
root@Khronicles-PC:~# ln app.log app_hard.log
root@Khronicles-PC:~# ls -al
total 52
drwx------  6 root root 4096 Mar  3 20:59 .
drwxr-xr-x 27 root root 4096 Mar  3 16:17 ..
-rw-------  1 root root 3682 Mar  1 22:32 .bash_history
-rw-r--r--  1 root root 3106 Apr 22  2024 .bashrc
-rw-------  1 root root   20 Feb 20 22:34 .lesshst
drwxr-xr-x  3 root root 4096 Feb 20 21:52 .local
-rw-r--r--  1 root root    0 Mar  3 13:44 .motd_shown
-rw-r--r--  1 root root  161 Apr 22  2024 .profile
drwx------  2 root root 4096 Feb 10 00:54 .ssh
-rw-r--r--  2 root root   12 Mar  3 20:58 app.log
-rw-r--r--  2 root root   12 Mar  3 20:58 app_hard.log
drwxr-xr-x  2 root root 4096 Mar  3 20:33 deploy
-rwxr-xr-x  1 root root   63 Mar  3 20:29 deploy.sh
drwxr-xr-x  2 root root 4096 Mar  3 20:28 src
root@Khronicles-PC:~# ln -s app.log app_soft.log
root@Khronicles-PC:~#
root@Khronicles-PC:~# rm app.log
root@Khronicles-PC:~#
root@Khronicles-PC:~# ls -l
total 16
-rw-r--r-- 1 root root   12 Mar  3 20:58 app_hard.log
lrwxrwxrwx 1 root root    7 Mar  3 20:59 app_soft.log -> app.log
drwxr-xr-x 2 root root 4096 Mar  3 20:33 deploy
-rwxr-xr-x 1 root root   63 Mar  3 20:29 deploy.sh
drwxr-xr-x 2 root root 4096 Mar  3 20:28 src
root@Khronicles-PC:~# cat app_hard.log
Log entry 1
root@Khronicles-PC:~# cat app_soft.log
cat: app_soft.log: No such file or directory
root@Khronicles-PC:~#
```

Explanation:

Inode Behavior

Hard links reference the same inode as the original file. Removing one filename does not delete the data until all hard links are removed.

Symbolic links reference the path (filename), not the inode. If the original file is deleted, the symlink becomes a dangling pointer.

Implications for Monitoring

Monitoring failure: If monitoring tools rely on symlinks, cleanup of the original file breaks the link, causing monitoring to fail.

Forensic reliability: Hard links preserve access to log data even if the original file is deleted, ensuring forensic continuity.

Scenario 6 – Sensitive Data Exposure Hunt

Grep –r

```
root@Khronicles-PC:~# grep -r "password" /var/log/
/var/log/bootstrap.log:Shadow passwords are now on.
/var/log/auth.log.1:2026-02-20T14:35:15.703099+00:00 Khronicles-PC passwd[420]: pam_unix(passwd:chauthtok): password changed for khroncles
/var/log/auth.log.1:2026-02-20T18:33:53.446340+00:00 Khronicles-PC passwd[1623]: pam_unix(passwd:chauthtok): password changed for audit_test
/var/log/auth.log.1:2026-02-20T20:02:01.630824+00:00 Khronicles-PC passwd[1762]: pam_unix(passwd:chauthtok): password changed for lateef
/var/log/auth.log.1:2026-02-20T20:02:23.885552+00:00 Khronicles-PC passwd[1781]: pam_unix(passwd:chauthtok): password changed for lateef
/var/log/auth.log.1:2026-02-20T20:03:16.346014+00:00 Khronicles-PC passwd[1818]: pam_unix(passwd:chauthtok): password changed for purity
/var/log/auth.log.1:2026-02-20T20:03:57.768601+00:00 Khronicles-PC passwd[1847]: pam_unix(passwd:chauthtok): password changed for lateef
/var/log/auth.log.1:2026-02-20T20:04:13.262604+00:00 Khronicles-PC passwd[1860]: pam_unix(passwd:chauthtok): password changed for purity
/var/log/auth.log.1:2026-02-20T20:04:59.193482+00:00 Khronicles-PC passwd[1869]: pam_unix(passwd:chauthtok): password changed for arthur
/var/log/auth.log.1:2026-02-20T20:05:33.568652+00:00 Khronicles-PC passwd[1878]: pam_unix(passwd:chauthtok): password changed for paula
/var/log/auth.log.1:2026-02-20T20:05:57.734910+00:00 Khronicles-PC passwd[1887]: pam_unix(passwd:chauthtok): password changed for yaa
/var/log/auth.log.1:2026-02-20T20:06:12.730595+00:00 Khronicles-PC passwd[1896]: pam_unix(passwd:chauthtok): password changed for alex
/var/log/auth.log.1:2026-02-20T20:06:54.958013+00:00 Khronicles-PC passwd[1913]: pam_unix(passwd:chauthtok): password changed for aminat
/var/log/auth.log.1:2026-02-20T20:35:27.980674+00:00 Khronicles-PC passwd[2068]: pam_unix(passwd:chauthtok): password changed for habiba
/var/log/auth.log.1:2026-02-27T19:20:33.632161+00:00 Khronicles-PC sudo: khroncles : 3 incorrect password attempts ; TTY=pts/0 ; PWD=/home/khroncles ; USER
c/shadow
/var/log/auth.log.1:2026-02-27T19:25:04.677499+00:00 Khronicles-PC sudo: khroncles : 3 incorrect password attempts ; TTY=pts/0 ; PWD=/home/khroncles ; USER
shadow
/var/log/auth.log.1:2026-02-27T19:28:04.307761+00:00 Khronicles-PC sudo: khroncles : 3 incorrect password attempts ; TTY=pts/0 ; PWD=/home/khroncles ; USER
/var/log/auth.log.1:2026-02-27T19:29:23.650268+00:00 Khronicles-PC passwd[1006]: pam_unix(passwd:chauthtok): password changed for khroncles
/var/log/auth.log.1:2026-02-27T19:39:13.581606+00:00 Khronicles-PC passwd[1153]: pam_unix(passwd:chauthtok): password changed for intern_a
/var/log/auth.log.1:2026-02-27T19:39:41.959235+00:00 Khronicles-PC passwd[1165]: pam_unix(passwd:chauthtok): password changed for dev_user
/var/log/auth.log.1:2026-02-27T19:40:08.993569+00:00 Khronicles-PC passwd[1167]: pam_unix(passwd:chauthtok): password changed for dev_user
/var/log/auth.log.1:2026-02-27T19:40:27.665512+00:00 Khronicles-PC passwd[1176]: pam_unix(passwd:chauthtok): password changed for ops_user
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/user-1000@00064bfab9955078-b7ceb7034fbb68bb.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064bd315e10ff5-73f2a797e8e8998d.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064bfab92e6717-000aa5cfeb4a578b.journal~: binary file matches
root@Khronicles-PC:~#
```

grep –r –e “password” –e “secret” –e “token” /etc/

```
root@Khronicles-PC:~# grep -r -e "password" -e "secret" -e "token" /etc/
/etc/ssl/openssl.cnf:# input_password = secret
/etc/ssl/openssl.cnf:# output_password = secret
/etc/ssl/openssl.cnf:challengePassword          = A challenge password
/etc/ssl/openssl.cnf:secret = pass:insta # can be used for both client and server side
/etc/ssl/openssl.cnf:secret = $insta::secret # pass:insta
/etc/ssl/openssl.cnf:secret = # disable PBM
/etc/cloud/templates/chef_client.rb.tmpl:{% if encrypted_data_bag_secret %}
/etc/cloud/templates/chef_client.rb.tmpl:encrypted_data_bag_secret "{{encrypted_data_bag_secret}}"
/etc/cloud/cloud.cfg:  - set_passwords
/etc/default/useradd:# The number of days after a password expires until the account
/etc/mime.types:application/token-introspection+jwt
/etc/mime.types:application/vnd.etsi.timestamp-token            tst
/etc/mime.types:application/vnd.marlin.drm.actiontoken+xml
/etc/mime.types:application/vnd.marlin.drm.conftoken+xml
/etc/apparmor.d/abstractions/ldapclient:  /etc/ldap.secret          r,
/etc/apparmor.d/abstractions/svn-repositories:  /srv/svn/**/db/lock-tokens     rwl,
/etc/apparmor.d/abstractions/authentication:  # databases containing passwords, PAM configuration files, PAM libraries
/etc/apparmor.d/1password:profile 1password /opt/1Password/1password flags=(unconfined) {
/etc/apparmor.d/1password:  include if exists <local/1password>
/etc/services:shell             514/tcp         cmd syslog      # no passwords used
/etc/debconf.conf:# World-readable, and accepts everything but passwords.
/etc/debconf.conf:Reject-Type: password
/etc/debconf.conf:# Not world readable (the default), and accepts only passwords.
/etc/debconf.conf:Name: passwords
/etc/debconf.conf:Accept-Type: password
/etc/debconf.conf:Filename: /var/cache/debconf/passwords.dat
/etc/debconf.conf:# databases, one to hold passwords and one for everything else.
/etc/debconf.conf:Stack: config, passwords
/etc/debconf.conf:# A remote LDAP database. It is also read-only. The password is really
/etc/debconf.conf:#BindPasswd: secret
/etc/login.defs:#       PASS_MAX_DAYS   Maximum number of days a password may be used.
/etc/login.defs:#       PASS_MIN_DAYS   Minimum number of days allowed between password changes.
/etc/login.defs:#       PASS_WARN_AGE   Number of days warning given before a password expires.
/etc/login.defs:# Max number of login retries if password is bad. This will most likely be
/etc/login.defs:# If set to "yes", new passwords will be encrypted using the MD5-based
```

Pattern file:

using >

```
root@Khronicles-PC:~# grep -r -f patterns.txt /etc/ > sensitive_results.txt
root@Khronicles-PC:~# cat sensitive_results.txt
/etc/ssl/openssl.cnf:# input_password = secret
/etc/ssl/openssl.cnf:# output_password = secret
/etc/ssl/openssl.cnf:secret = pass:insta # can be used for both client and server side
/etc/ssl/openssl.cnf:secret = $insta::secret # pass:insta
/etc/ssl/openssl.cnf:secret = # disable PBM
/etc/cloud/templates/chef_client.rb.tmpl:{% if encrypted_data_bag_secret %}
/etc/cloud/templates/chef_client.rb.tmpl:encrypted_data_bag_secret "{{encrypted_data_bag_secret}}"
/etc/default/useradd:# The number of days after a password expires until the account
/etc/mime.types:application/token-introspection+jwt
/etc/mime.types:application/vnd.etsi.timestamp-token            tst
/etc/mime.types:application/vnd.marlin.drm.actiontoken+xml
/etc/mime.types:application/vnd.marlin.drm.conftoken+xml
/etc/apparmor.d/abstractions/ldapclient:  /etc/ldap.secret          r,
/etc/apparmor.d/abstractions/svn-repositories:  /srv/svn/**/db/lock-tokens     rwl,
/etc/apparmor.d/1password:profile 1password /opt/1Password/1password flags=(unconfined) {
/etc/debconf.conf:# A remote LDAP database. It is also read-only. The password is really
/etc/debconf.conf:#BindPasswd: secret
/etc/login.defs:#       PASS_MAX_DAYS   Maximum number of days a password may be used.
/etc/login.defs:#       PASS_MIN_DAYS   Minimum number of days allowed between password changes.
/etc/login.defs:#       PASS_WARN_AGE   Number of days warning given before a password expires.
/etc/login.defs:# Max number of login retries if password is bad. This will most likely be
/etc/login.defs:# If set to DES, DES-based algorithm will be used for encrypting password (default)
/etc/security/group.conf:# namely individual tokens that are optionally prefixed with '!' (logical
/etc/security/time.conf:# namely individual tokens that are optionally prefixed with '!' (logical
/etc/sudo_logsrvd.conf:# password in the terminal input using passprompt_regex.
/etc/sudo_logsrvd.conf:# password prompts in the terminal output when log_passwords is
/etc/pam.d/common-password:# /etc/pam.d/common-password - password-related modules common to all services
/etc/pam.d/common-password:#used the option "sha512"; if a shadow password hash will be shared
/etc/sudo.conf:# password prompt for "sudo -A" support.  Sudo does not ship with its
root@Khronicles-PC:~#
```

Using >>

```
root@Khronicles-PC:~# grep -r -f patterns.txt /var/log/ >> sensitive_results.txt
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/user-1000@00064bfab9955078-b7ceb7034fbb68bb.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064bd315e10ff5-73f2a797e8e8998d.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@afd104e0d89a4186b2fa86fb82bb9e24-0000000000001714-00064c1ed477d8ed.journal: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064c1af63d5c00-5ac3502e08b04a17.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064c1ed478bf99-f15d4d2bb5a58a3c.journal~: binary file matches
grep: /var/log/journal/e8fb883388de4d5aa0e16f4c28ef6b08/system@00064bfab92e6717-000aa5cfeb4a578b.journal~: binary file matches
root@Khronicles-PC:~# cat sensitive_results
cat: sensitive_results: No such file or directory
root@Khronicles-PC:~# cat sensitive_results.txt
/etc/ssl/openssl.cnf:# input_password = secret
/etc/ssl/openssl.cnf:# output_password = secret
/etc/ssl/openssl.cnf:secret = pass:insta # can be used for both client and server side
/etc/ssl/openssl.cnf:secret = $insta::secret # pass:insta
/etc/ssl/openssl.cnf:secret = # disable PBM
/etc/cloud/templates/chef_client.rb.tmpl:{% if encrypted_data_bag_secret %}
/etc/cloud/templates/chef_client.rb.tmpl:encrypted_data_bag_secret "{{encrypted_data_bag_secret}}"
/etc/default/useradd:# The number of days after a password expires until the account
/etc/mime.types:application/token-introspection+jwt
```

Explanation:

The reason both > and >> are used is because > is used to create a fresh result file while >> is used to append new data to the existing file without clearing out the old data in the file.

Scenario 7: Shared Directory Stability Controls

```
root@Khronicles-PC:~# mkdir /shared_dev
root@Khronicles-PC:~# chown :project_team /shared_dev
root@Khronicles-PC:~# chmod 1777 /shared_dev
root@Khronicles-PC:~#
```

Explanation:

Mechanism – Sticky Bit

The sticky bit is a critical control when applied on a directory enables users to create files, delete or rename only their own files and prevents the deletion or renaming of files owned by others.

Security Reasoning:

Without the sticky bit, any user with write access to the directory could delete or rename files created by others.

With the sticky bit, file ownership is respected: only the owner (or root) can remove their files.